Senior Project Review Report

Use this document as a guideline to write your report.

How to use this document

Use this as a template and checklist. Your final report should tell a complete engineering story: what problem you solved, what you built, why the design choices make sense, and evidence that it meets requirements.

Recommended report length

• 20 pages maximum, but also depending on project scope
• Use appendices for detailed schematics, full code listings, and raw datasets

1. Report structure (recommended)

Section	What to include
Front matter	Title page, abstract/executive summary, TOC, list of figures/tables (optional).
1. Introduction	Problem statement, motivation, stakeholders, constraints, and project objectives.
2. Requirements and success criteria	Functional + non-functional requirements, measurable acceptance criteria, and traceability.
3. System overview	High-level architecture: block diagram, subsystems, interfaces, power domains, data flow.
4. Detailed design	Hardware, firmware/software, algorithms, mechanical (if any). Key decisions and tradeoffs.
5. Implementation	Build details: PCB, assembly, tooling, version control, development environment, prototype building.
6. Verification and validation (V&V)	Test plan, procedures, equipment, and results mapped back to requirements.
7. Results and discussion	Performance metrics, plots, limits, failure modes, what you learned, and what you would improve.

8. Project management	Schedule, milestones, budget/BOM, risks, mitigations, and changes over time.
9. Conclusion and continue next semester	What worked, what didn’t, and the next steps for next semester.
References	Datasheets, standards, papers, and online resources.
Appendices	Schematics, BOM, test logs, code excerpts, additional plots, safety notes, etc.

2. Writing principles reviewers care about

- Start from requirements: every design choice should connect to a requirement or constraint.
- Be explicit about assumptions, operating conditions, and units (voltage, current, bandwidth, temperature, etc.).
- Show evidence: measured data beats claims. Include plots, tables, and photos when helpful.
- Tell the tradeoff story: cost vs. performance, power vs. noise, complexity vs. robustness.
- Make it reproducible: include firmware versions, PCB revision, test equipment models, and configuration settings.
- Write for skimming: clear headings, short paragraphs, captions that explain figures without reading the body text.

3. Abstract / Executive summary (1 page)

Write this last, but place it first. In 200–400 words, answer:

- What problem did you solve and why does it matter?
- What did you build (system-level description)?
- What are the key results (top 3 metrics) and do they meet requirements?
- What is novel or challenging about your design?
- What remains unfinished this term or to be continued next term?

Include a simple snapshot figure if it helps (system photo or block diagram).

4. Requirements and success criteria

Use a requirements table with measurable acceptance criteria. Avoid vague terms like “fast”, “low power”, or “high accuracy” without numbers, tolerances, and test conditions.

Minimum recommended requirement categories:

- Functionality
- Performance
- Power/energy
- Safety
- Reliability/robustness
- Usability
- Cost
- Compliance/standards (if applicable)

Requirements table template

Req #	Requirement	Acceptance criteria (measurable)	Method (analysis/test/inspect)	Priority	Status (Pass/Fail)
List all requirements for your deliverables.					

5. System overview and architecture

Include a block diagram and clearly define interfaces between subsystems.

- Block diagram with labeled signals, voltage rails, and communication links (I2C, SPI, UART, CAN, Ethernet, BLE, etc.).
- Operating modes and state diagram (if modes exist).
- Data flow: where data is sensed, processed, stored, and transmitted.
- Power tree: sources, regulators, expected current draw by subsystem.
- Key constraints: size, weight, environment, timing, safety limits.

6. Detailed design

Organize by subsystem. For each, include intent, design, and rationale.

6.1 Hardware design

- Schematic excerpts with callouts (don’t paste unreadable full-page schematics in the main body).
- Component selection rationale (parts availability, ratings, cost, efficiency, noise, tolerance).
- Key calculations: resistor dividers, op-amp gain/bandwidth, filter cutoff, thermal dissipation, battery life, etc.
- PCB considerations: stackup, grounding strategy, isolation, clearance, decoupling.
- Safety considerations: fusing, current limiting, ESD, isolation, enclosure, labeling.

6.2 Firmware / software design

- Architecture diagram (tasks/threads/ISR, state machines, modules).
- Interfaces and protocols (packet formats, timing diagrams, error handling).
- Configuration management (GitHub workflow, branching, releases, issue tracking).
- Testing approach (unit tests, hardware-in-the-loop, simulation).

6.3 Algorithms / signal processing (if applicable)

- **Mathematical model and assumptions. (MUST BE INCLUDED.)**
- Training/data collection methodology (if ML) and dataset splits.
- Computational complexity and memory requirements.
- Performance metrics and why they match the problem (accuracy, latency, SNR, etc.).

7. Implementation and build

- Prototype iterations: what changed between revisions and why.
- Assembly process: soldering, reflow, rework, programming steps, calibration.
- Bill of materials (BOM): include vendor part numbers and unit costs (can be in appendix).
- Manufacturing constraints: lead times, DFM notes, enclosure/fixture fabrication.
- Photos: annotated PCB, enclosure, wiring harness, test setup.

8. Verification and validation (V&V)

Verification asks: Did we build the prototype right? Validation asks: Did we build the right prototype? Your V&V section should map tests to requirements and include evidence.

V&V matrix template

Test #	Related Req #	Test objective	Procedure summary	Equipment / setup	Pass criteria	Result / link to figure

For each **major test**, include:

- Test setup diagram/photo, with key settings (supply voltage, load, sampling rate).
- Uncertainty/limitations (instrument accuracy, calibration, environmental effects).
- Representative plots and summary statistics (mean, min/max, standard deviation).
- Failure cases and debugging notes.

9. Results and discussion

Compare results to requirements and explain gaps.

- Top-line performance table (key metrics vs. targets).
- Plots with labeled axes, units, and legends; annotate operating points.
- Energy/power results (average, peak, duty cycle) with assumptions.
- Tradeoff analysis (what you optimized for, what you sacrificed, and why).
- Lessons learned: top 3 engineering lessons that would help next term.

10. Project management (schedule, budget, risks)

10.1 Schedule and milestones

Include a timeline (Gantt or milestone list) and note major changes from the original plan.

Suggested milestone breakdown: requirements freeze → architecture review → prototype 1 → prototype 2 → integration → V&V → final demo/report.

10.2 Budget / BOM

Provide both a summary budget and a detailed BOM.

Budget summary template

Category	Item(s)	Qty	Unit cost	Total cost
Electronics	[MCU, sensors, passives]	—	—	—
Fabrication	[PCB, stencil, enclosure]	—	—	—

11. Conclusion and next term work

- Summarize whether you met the requirements (brief) for this term.
- State the single biggest technical contribution.
- List 3–5 concrete next steps (not vague).
- Document what you would do differently if starting over for this term.

12. References

Cite sources for any non-trivial fact: datasheets, application notes, standards, and algorithms.

At minimum, cite:

- All IC/part datasheets and application notes used in the design
- Standards (USB, IEC, FCC, IEEE, etc.) if relevant
- Open-source libraries/frameworks (with version and license)
- Papers or tutorials for algorithms, filters, control laws, or ML models

13. Appendices (suggested)

- Full schematics (legible), PCB layout renders, stackup details
- Full BOM with supplier links and alternates
- Calibration and test logs (raw data)
- Firmware build instructions and configuration files
- Additional plots, corner-case tests, and troubleshooting notes
- Individual module test result
- References are complete and consistent.